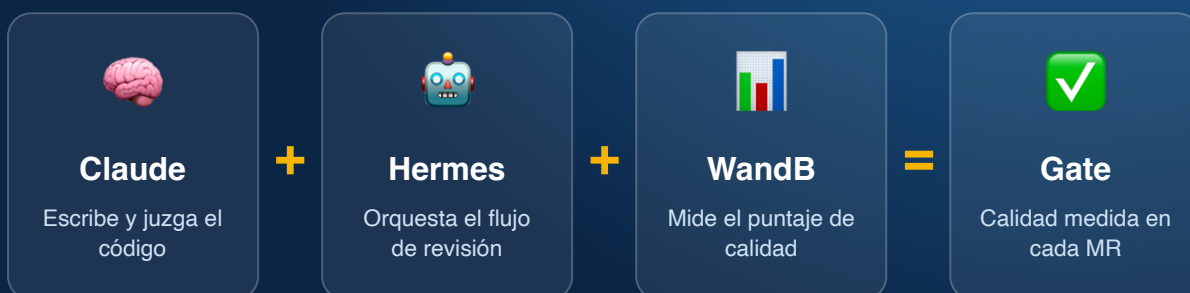


IA aplicada al desarrollo

Cómo usamos Hermes, Weights & Biases y Claude
para subir la calidad y la velocidad del equipo



Repos: `adjustments-silin-db` · `adjustments-silin-api`

Rama: `feature/ajustes-manuales-IA` · WI 54929 — Ajustes Manuales Autogestionados

Documento guía para el equipo · Junio 2026

El objetivo, en una imagen

PROPÓSITO PRINCIPAL

Todo el código —**base de datos, API y frontend**— lo genera la IA bajo **estricta rigurosidad**: cada cambio se mide y se aprueba antes de fusionarse. Para el ejemplo arrancamos desde el nivel más bajo: la **base de datos**.



Frontend · si aplica



API · Go



Base de datos · DDL

[el ejemplo de esta guía ▶](#)



Misma vara de calidad medida en cada capa · el ejemplo arranca por la base.



Desarrollador abre un Merge Request con un cambio de DDL



HERMES — orquesta el flujo

El job `job_hermes_gate` corre tres subagentes en cadena

`judge.py`

puntúa cada changeset 0–100 sobre la rúbrica

 Claude juzga

`comment_mr.py`

publica la tabla-veredicto como comentario en el MR

`gate.py`

compara la media contra el umbral 0.85



PASS — media ≥ 0.85 · puede fusionarse



BLOCK — media < 0.85 · mejora el changeset y vuelve a empujar



Calibración previa (fase exploratoria): antes de productizar el gate, **WandB** midió y graficó el puntaje de cada categoría x modelo x spec sobre 13 corridas. De ahí salieron la **rúbrica de 6 categorías** y el **umbral 0.85** que usa `gate.py`.



Claude — modelo que juzga/genera



Hermes — orquestación



WandB — medición

Parte 1 — Resumen ejecutivo

Para líderes, PO y cualquiera que quiera entender el *qué* y el *porqué* sin entrar al código.

El problema

Los cambios de base de datos (DDL) son caros de deshacer en producción. La revisión humana es lenta, inconsistente y depende de quién revise y de su cansancio. Al mismo tiempo, un agente de IA que no conoce las convenciones del repo inventa nombres, rompe patrones y consume tokens (= dinero) en cada mensaje.

Lo que construimos

1. Compuerta de calidad con IA

Cada cambio de esquema recibe una **puntuación automática y objetiva** (0–100) antes de poder fusionarse. Disponible hoy en `adjustments-silin-db`.

2. Contexto curado para la IA

Documentación (`CLAUDE.md` , `GOTCHAS.md` , `CORE_LIB.md`) + memoria persistente que hacen que la IA acierte más y gaste menos.

3. Decisiones medidas, no a ojo

Experimentos rastreados en WandB que respondieron: *¿qué modelo usar y cuánto invertir en el prompt?* con datos, no intuición.

Resultado real del piloto (MR !58)

El juez de IA (`claude-opus-4-7`) puntuó los 4 changesets DDL de WI 54929:

Changeset	Puntaje /100	Precisión
<code>00010_create_adjustment_batch_record</code>	94	0.94
<code>00001_create_adjustment_batch_record_indexes</code>	93	0.93
<code>00001_create_adjustment_batch_status_type</code>	89	0.89
<code>00009_create_adjustment_batch</code>	88	0.88

Veredicto: precisión media `0.91` $\geq$ umbral `0.85` →  **PASS**. Comentario publicado automáticamente en el MR (nota 114476), 5 artefactos subidos.

Por qué importa

- **Atrapa DDL riesgoso antes de tocar una base de datos.** Red de seguridad barata pre-merge.
- **Mismo estándar, todos los MR, todos los autores.** Sin fatiga de revisor, sin "a mí me parece bien".
- **Revisiones más rápidas.** El humano arranca de un resumen puntuado, no de un diff frío.
- **Costo casi nulo.** Una llamada de IA por changeset, salida diminuta → centavos por MR.

Lo que NO hace: no reemplaza la revisión humana ni ejecuta migraciones. Es una señal de calidad que *aumenta* el pipeline de Liquibase existente. Hoy es **advisory** (avisa pero no bloquea) mientras el equipo calibra.

Las tres herramientas en una línea

Herramienta	Qué es	Para qué la usamos
Claude / Claude Code	Modelo de IA de Anthropic + su CLI agente en el terminal.	Escribir y revisar DDL, handlers Go y tests; actuar como juez del gate; documentar.
Hermes	Harness propio (de hermes-exploratory) que orquesta el flujo : subagentes que juzgan, comentan y aplican la compuerta.	El motor que coordina la compuerta de calidad de DDL.
WandB (Weights & Biases)	La capa de medición. Registra y grafica el puntaje final de cada categoría x modelo x spec.	Medir objetivamente qué modelo + spec puntúa mejor en cada categoría de la rúbrica.

Parte 2 — Playbook práctico

Para ingenieros que quieran *entender a fondo* y *reproducir* esto en otro repo.

0 · Antes de empezar — entorno, llaves y MCPs

El flujo se apoya en unas pocas herramientas locales, una llave de API y los servidores MCP que le dan a Claude acceso a tus sistemas (repos, work items, diseños). Instala/configura solo lo que tu repo necesite.

Herramientas a instalar

Herramienta	Para qué	Nota
Claude Code (CLI)	El agente de IA en el terminal: escribe, revisa, documenta.	Requiere ANTHROPIC_API_KEY (abajo).
Python 3.11+	Corre los scripts del gate (<code>judge.py</code> , <code>comment_mr.py</code> , <code>gate.py</code>).	<code>pip install -r ci/hermes-gate/requirements.txt</code> .
Docker + docker-compose	Pruebas de BD en local: levanta Liquibase + PostgreSQL.	No necesitas instalar Liquibase aparte (corre vía imagen <code>liquibase/liquibase:4.28</code>).
git + g <code>lab</code> / <code>gh</code>	Operaciones de repo; <code>g<code>lab</code></code> para GitLab, <code>gh</code> para GitHub.	Opcional — el MCP cubre casi todo; útil para re-correr el gate empujando un commit.

Llaves y secretos

Variable	Para qué	Dónde
ANTHROPIC_API_KEY	Claude (local) + el juez del gate (CI).	Env local y variable CI/CD — masked, NO protected .
GITLAB_TOKEN	Token con scope <code>api</code> para que el gate comente en el MR.	Variable CI/CD — masked, NO protected .

Seguridad: nunca commitees llaves. Van en variables de entorno / variables CI / gestor de secretos, nunca en el código. Rota cualquier llave que se haya podido exponer.

Servidores MCP (configurar según necesidad)

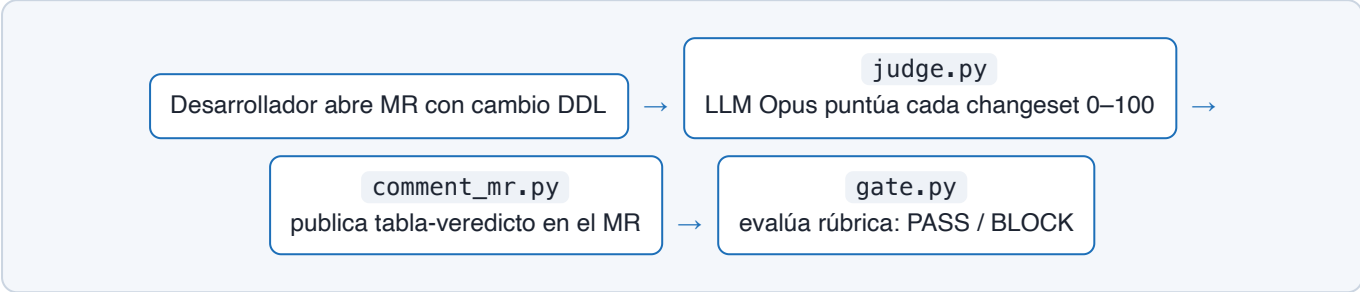
Un MCP le da a Claude un conjunto de herramientas para hablar con un sistema externo. Se configuran con `/mcp` (autenticación por servidor: OAuth o token). Activa solo los que uses:

MCP	Para qué en este proyecto
GitLab	Leer/escribir repos, MR y notas. Los repos <code>adjustments-silin-db</code> y <code>-api</code> viven aquí.
GitHub	Repos/PRs externos — p. ej. el upstream <code>hermes-exploratory</code> .
Azure DevOps	Leer el PRD: WI 54929 vive aquí (<code>get_work_item</code>).
Figma	Referencia UX/UI del módulo de ajustes (para la fase de frontend posterior).
Google Drive	Adjuntos del PRD (Google Doc + diagrama <code>diagrams.net</code> enlazados en el WI).
context7	Documentación actualizada de librerías/frameworks (evita adivinar APIs).

Regla práctica: empieza con GitLab + Azure DevOps (repos + PRD). Suma Figma cuando abra la fase FE, y context7 al integrar librerías nuevas.

1 · La compuerta Hermes de DDL — cómo funciona

En cada Merge Request que toca `changelog/**/*.*.sql`, `rollbacks/**/*.*.sql` o `ci/hermes-gate/**`, el job `job_hermes_gate` corre **tres scripts en cadena**:



Si el MR no toca DDL, el gate se salta y el pipeline queda verde. El job es independiente de los jobs de Liquibase (no usa credenciales de BD).

La rúbrica (100 puntos, definida en `rubric.py`)

Categoría	Máx	Qué puntúa
structure	25	Un CREATE/ALTER/DROP por archivo; cabecera <code>--> ... <--</code> ; SQL limpio.
naming	20	<code><table>_pk</code> , <code>fk_<child>_<parent></code> , <code>uk_*</code> , <code>idx_*</code> , <code>snake_case</code> .
integrity	20	Columnas de auditoría NOT NULL, PK UUID, dinero <code>NUMERIC(20,2)</code> , ENUM para ciclo de vida.
rollback	15	Rollback presente y que <i>revierte</i> el changeset.
changelog_wiring	10	Referenciado en <code>00000_changelog.yaml</code> con <code>id/author/context/labels/comment</code> .
spec_adherence	10	<code>labels</code> : coincide con WI/HU del MR; entidades alinean con la descripción.

Precisión = `total / 100`. El gate compara la **media de precisión de todos los changesets cambiados** contra el umbral `0.85`.

Cómo leer el comentario

Cada changeset tiene una fila con los 6 sub-puntajes, un total y su precisión. El pie muestra la media y el veredicto **PASS** / **BLOCK**. Un sub-puntaje bajo dice *exactamente* qué arreglar (ej: `naming` de 13 = identificadores poco claros o inconsistentes).

Cómo re-correr el gate

```
git commit --allow-empty -m "re-run gate" && git push
```

Gotcha: el token de GitLab usado por `glab /MCP` *no puede* reintentar jobs de CI (`403 insufficient_scope`). Re-correr se hace empujando un commit, no reintentando el job desde la API.

2 · Setup (una sola vez) — variables de CI

El gate necesita dos variables CI/CD (Settings → CI/CD → Variables):

Variable	Para qué	Masked	Protected
ANTHROPIC_API_KEY	Permite al juez llamar a la IA. Sin ella, el job se salta en verde.	✔ Sí	✗ No
GITLAB_TOKEN	Token con scope <code>api</code> para publicar el comentario en el MR.	✔ Sí	✗ No

⚠ **Gotcha crítico — NO marcar "Protected".** Una variable protegida solo se expone a jobs en *ramas protegidas*. Las ramas feature no lo son, así que una variable protegida es invisible ahí y el job se salta en silencio (verde, pero sin revisar nada). Dejar **Protected sin marcar**. Verificado en vivo en MR !58.

Diseño a prueba de fallos: si falta la key, el job sale con `exit 0` (advisory) y el pipeline queda verde en vez de fallar.

De advisory a bloqueante

Hoy: `allow_failure: true` en `.gitlab-ci.yml` — un puntaje bajo avisa pero no detiene el merge. Cuando el equipo termine de calibrar:

```
job_hermes_gate:
  allow_failure: false  # antes: true
```

y opcionalmente agregarlo como *required check* en la protección de rama. Después de eso, un MR por debajo de `0.85` de media no podrá fusionarse hasta mejorar los changesets.

3 · La regla de oro (hallazgo de fase-1)

Si la salida de la IA puntúa por debajo de 80, mejora el SPEC, no el modelo. Escribir un mejor spec resultó **3x más efectivo** que subir de modelo. Reflejo de subir de Sonnet a Opus con un spec pobre = desperdicia dinero y rara vez arregla el problema de fondo.

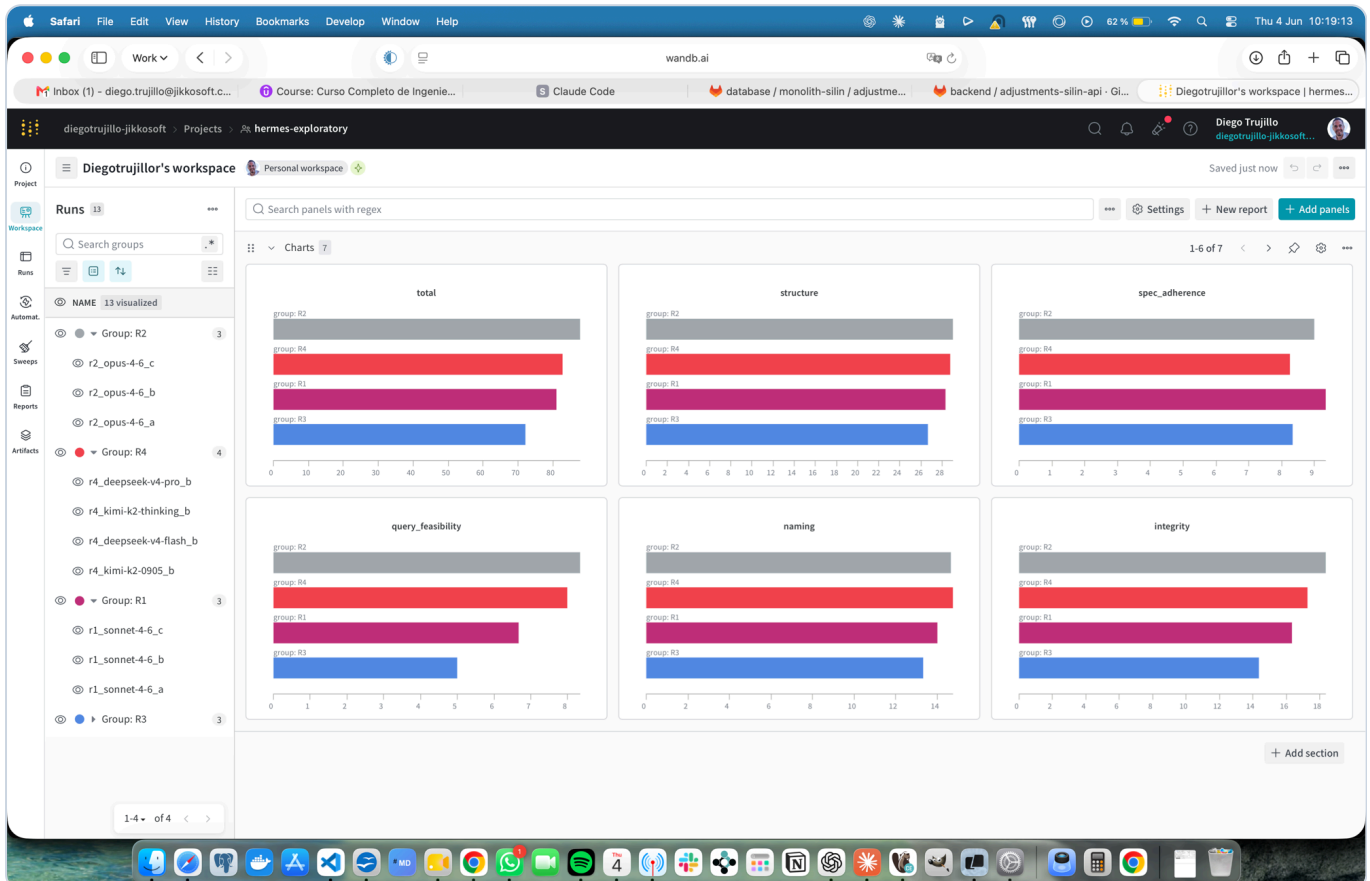
Este hallazgo salió de un experimento "profundidad de spec x nivel de modelo" (13 corridas) cuyos puntajes se **midieron y graficaron en WandB** (ver sección 4). De ahí salieron nuestras reglas de selección de modelo:

Tipo de tarea	Modelo por defecto	Por qué
Borrador/revisión de DDL	claude-sonnet-4-6	86 pts en spec balanceado; ~3x más barato que Opus por -5 pts.
DDL de cumplimiento / dinero (ej. WI 54929)	claude-opus-4-7	+5 pts de margen; justificado cuando el cambio toca dinero.
Boilerplate repetitivo, búsquedas	claude-haiku-4-5	80 pts — aceptable cuando un buen spec/skill carga las convenciones.

4 · WandB — la capa de medición

Hermes *orquesta* el flujo (subagentes que juzgan, comentan y aplican la compuerta), pero el **puntaje final de cada categoría, modelo y spec lo mide y registra WandB** (Weights & Biases). Cada corrida del experimento se loguea como un *run*; WandB agrupa los runs y grafica el puntaje por categoría de la rúbrica, lo que permite comparar combinaciones objetivamente — no "a ojo".

Reparto de roles: **Hermes** = orquestación (juez + comentario + gate). **WandB** = medición (scores por categoría × modelo × spec, graficados). **Claude** = el modelo que juzga y genera. La compuerta de CI es la versión productizada de lo que aquí se midió.



Workspace WandB · proyecto `hermes-exploratory` · 13 runs agrupados R1–R4. Cada panel es una categoría de la rúbrica (`total` , `structure` , `spec_adherence` , `query_feasibility` , `naming` , `integrity` , ...). Cada barra = puntaje de un grupo `specxmodelo` (ej. `r1_sonnet-4-6` , `r2_opus-4-6` , `r4_deepseek-v4-pro` , `r4_kimi-k2-thinking`). Así se ve, de un vistazo, qué combinación gana en cada categoría.

Cómo leer los paneles

- **Un panel por categoría.** El panel `total` es la suma /100; los demás son las sub-categorías de la rúbrica.
- **Una barra por grupo (`specxmodelo`).** Grupos R1–R4 = variantes de spec/modelo; el sufijo `_a/_b/_c` son repeticiones para medir consistencia.
- **Comparación directa.** Barras más largas = mejor puntaje. Aquí es donde se vio que un spec más profundo sube el puntaje más que cambiar de modelo — la evidencia detrás de la *regla de oro* de la sección 3.

5 · Documentación como contexto para la IA

Tres archivos, un objetivo: que la IA acierte sin reinventar, gastando los mínimos tokens.

Archivo	Analogía	Contenido
CLAU-DE.md	Portada corta, siempre a la vista	Reglas pequeñas <i>always-on</i> + punteros ("lee GOTCHAS antes de editar handlers").
GOT-CHAS.md	Lista de trampas	Cosas que te rompen si no las sabes de antemano. Ej: "este parser de Excel espera 21 columnas pero la plantilla tiene 9 — mal". Leer antes de tocar código.
CORE_-LIB.md	Chuleta de la librería compartida	Qué ofrece <code>core-silin-lib</code> (server, router, logger, DB, auth) para no reinventar ni adivinar. Leer antes de cablear algo nuevo.

Por qué separar de CLAUDE.md : CLAUDE.md se carga en *cada* turno → cuesta tokens en cada mensaje. Mantenerlo pequeño = más barato y rápido. GOTCHAS / CORE_LIB son grandes y solo importan a veces (al editar código específico), así que se cargan *bajo demanda*. Misma idea que: portada corta, manuales en el cajón, agarrar el manual solo al hacer ese trabajo. Ahorra presupuesto de contexto.

Memoria persistente (claude-mem): decisiones, gotchas, contexto de WI y aprobaciones se registran apenas ocurren, en archivos (uno por entrada + índice MEMORY.md). Así ninguna sesión arranca "sin memoria". Regla: antes de recomendar algo que nombra un archivo/función desde la memoria, verificar que aún existe en el árbol de trabajo (la memoria es una afirmación sobre el pasado, no un hecho del presente).

6 · Pruebas de BD en local (sin tocar ambientes)

Antes de abrir el MR, los changesets se prueban en local con Docker: **Liquibase + PostgreSQL** (`docker-compose up`). El pipeline real reproduce el flujo en CI: `snapshot → validate → status → update (manual) → history → db_doc`, seleccionando credenciales por rama (`develop→dev`, `qa→qa`, `staging→staging`, `main→prod`).

7 · WI 54929 — PDF de resolución (M2M)

Cumpliendo el user story, cada acción efectuada genera un **PDF de resolución**. Mencionar **M2M** (machine-to-machine) en los casos de: procesamiento en lote, o marcado como error pero con movimiento financiero aplicado, etc. El ciclo de vida por registro es `cargado → (aprobado | rechazado) → procesado`; los rechazados quedan *excluidos de forma efectiva* (no van al motor, no generan movimientos, no se marcan como procesados).

8 · Cómo llevarlo a tu repo (receta)

- 1 Copia `ci/hermes-gate/` (rubric, judge, gate, comment_mr, config.yml, requirements.txt).
- 2 Reescribe las 6 categorías de `rubric.py` + los checklists de `judge.py` para las convenciones de *tu* repo.
- 3 Agrega el stage `job_hermes_gate` a tu `.gitlab-ci.yml` (regla: corre en MR que tocan tus archivos clave; `allow_failure: true` al inicio).
- 4 Configura `ANTHROPIC_API_KEY` y `GITLAB_TOKEN` como variables CI — **masked, NO protected**.

5. **5** Calibra: corre contra un corpus *bueno* conocido (espera ≥ 0.85) y uno *mal* a propósito (espera < 0.50). Ajusta el prompt del juez si fallan.
6. **6** Cuando ambas calibraciones concuerden con la rúbrica, voltea a `allow_failure: false` y márcalo como required check.

Recordatorio final: cuando algo salga mal, mejora el spec/changeset (buenos comentarios, constraints explícitos, rollback que coincide) antes de cambiar de modelo. Un changeset más claro puntúa más alto de forma mucho más confiable.

Glosario y referencias

- **DDL** — Data Definition Language: SQL que crea/altera/borra estructura (tablas, índices, tipos).
- **Changeset** — unidad atómica de cambio de esquema en Liquibase.
- **LLM-as-judge** — usar un modelo de lenguaje como evaluador con rúbrica fija.
- **M2M** — machine-to-machine: flujo automatizado sin intervención humana.
- **Advisory** — el gate avisa pero no bloquea el merge (estado actual).
- **WandB** (Weights & Biases) — plataforma de tracking de experimentos; aquí mide y grafica los puntajes por categoría x modelo x spec.
- **Run / Group** (WandB) — un *run* es una corrida del experimento; un *group* junta runs de la misma variante specxmodelo.
- Guía completa del gate: `docs/hermes-ddl-gate.md` (en `adjustments-silin-db`).
- Upstream del harness: `hermes-exploratory` (fases 1 y 2).

IA aplicada en SILIN · Hermes · WandB · Claude — documento guía para el equipo. Generado a partir de los repos `adjustments-silin-db` y `adjustments-silin-api`, rama `feature/ajustes-manuales-IA`. Las cifras (MR !58, pipeline 83503) reflejan el piloto verificado en vivo.